

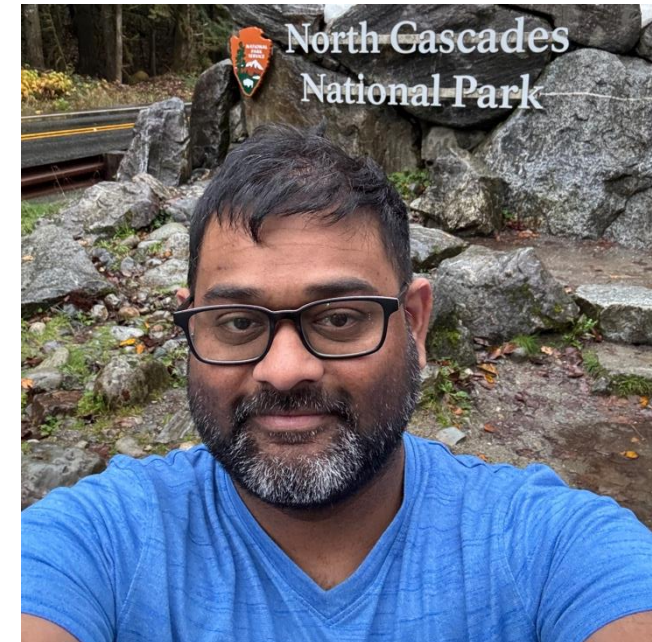
Beyond Read Replicas: Introducing QueryPilot for MySQL Scaling

October 4th, 2025

São Paulo



Learnings from
building data & AI
platforms for nearly
two decades



Agenda

1. Challenge of scaling databases
2. Common solutions
3. Limitations
4. Readysset QueryPilot
5. Demo

Scaling databases is an ongoing challenge

- It is not uncommon to see degradation once tables hit millions of rows as even the most basic database operations become expensive.
- Many SaaS applications find new “slow queries” arising in production only after adoption grows, forcing schema rewrites midlife.
- Schema changes make queries harder to optimize, leading to unpredictable performance under scale.

*“SQL Query was **running fine on 100K** rows but as soon as we **hit 2M+ rows** in the table, the **performance just went to s*****”*

Scaling databases is an ongoing challenge

- Disk I/O, network bandwidth, buffer pool pressure, and contention all rise with scale.
- Many deployments see storage I/O latency impacting throughput before CPU cores are saturated.
- Several studies done on database scaling[1] often identify infrastructure issues such as coordination, I/O bottlenecks, and network as primary pain points in scaling out relational systems.

*“...keep increasing the resources on my EC2 instance but it **is no longer helping** with query throughput or performance. It is almost like I am at **a point of diminishing returns!**...”*



Scaling databases is an ongoing challenge

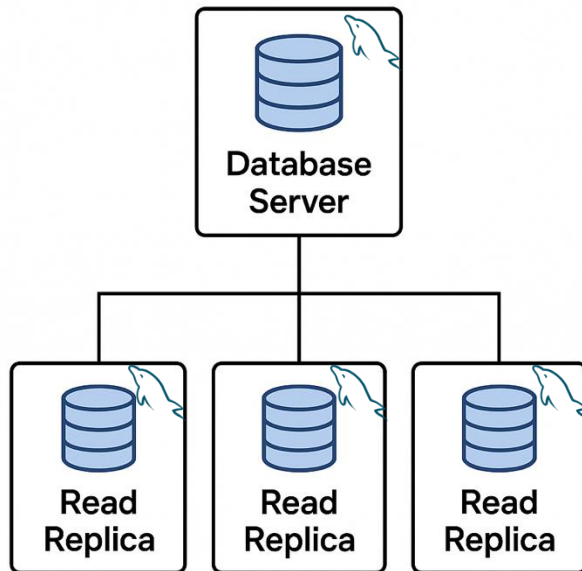
- More servers → more failure modes, more configuration drift, more things to monitor, upgrade, and troubleshoot.
- Even small changes to schema, index, partition become harder when database grows

*“We have recently grown to hundreds of customers. I had to add dozens of database nodes. Now I am **dealing with this “DevOps Tax”** to keep everything consistent and performant.”*

“Growing AWS Aurora invoice keeps me up at night.”



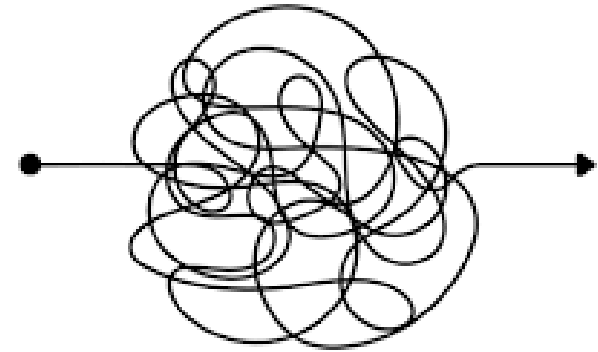
How do teams address the scaling challenges?



Read Replicas... Mostly 😊

Queries keep get slower as the data grows

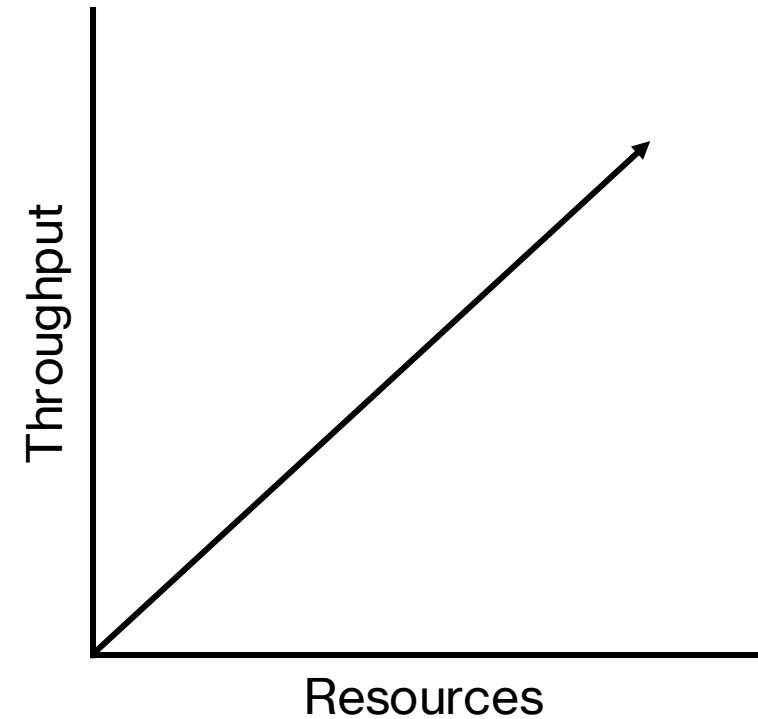
1. Replicas contain the same schema and dataset as the primary, so inefficient queries remain inefficient.
2. A poorly tuned JOIN, sort, or index scan doesn't magically run faster. They are just running on another box.
3. Replicas don't optimize queries, indexes, or execution plans.



Query

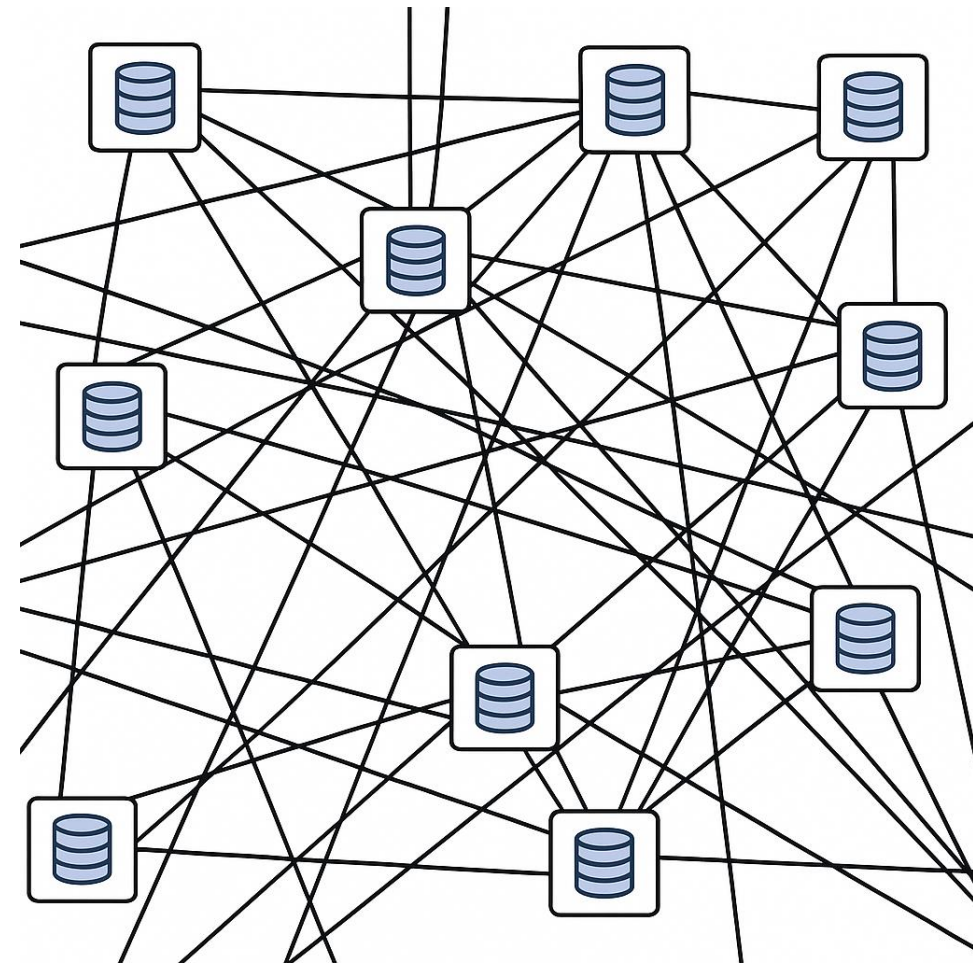
Increased I/O and Network Pressure

1. Replicas adds CPU, I/O, and storage overhead. Period.
2. Replication traffic consumes network bandwidth; cross-region replicas magnify this cost.
3. In a recent study[1] of replication overhead across multiple databases, adding two replicas increased total network traffic by as much as **300%** compared to having no replicas.



Operational Complexity Multiplies

1. More replicas = more moving parts: backups, monitoring, schema changes, and failover all get harder.
2. Failover isn't trivial: picking the "right" replica is a hard problem and can be risky as you can have data loss or inconsistencies.
3. Every replica is another full database to maintain. More patching, scaling, and debugging multiply with each node.



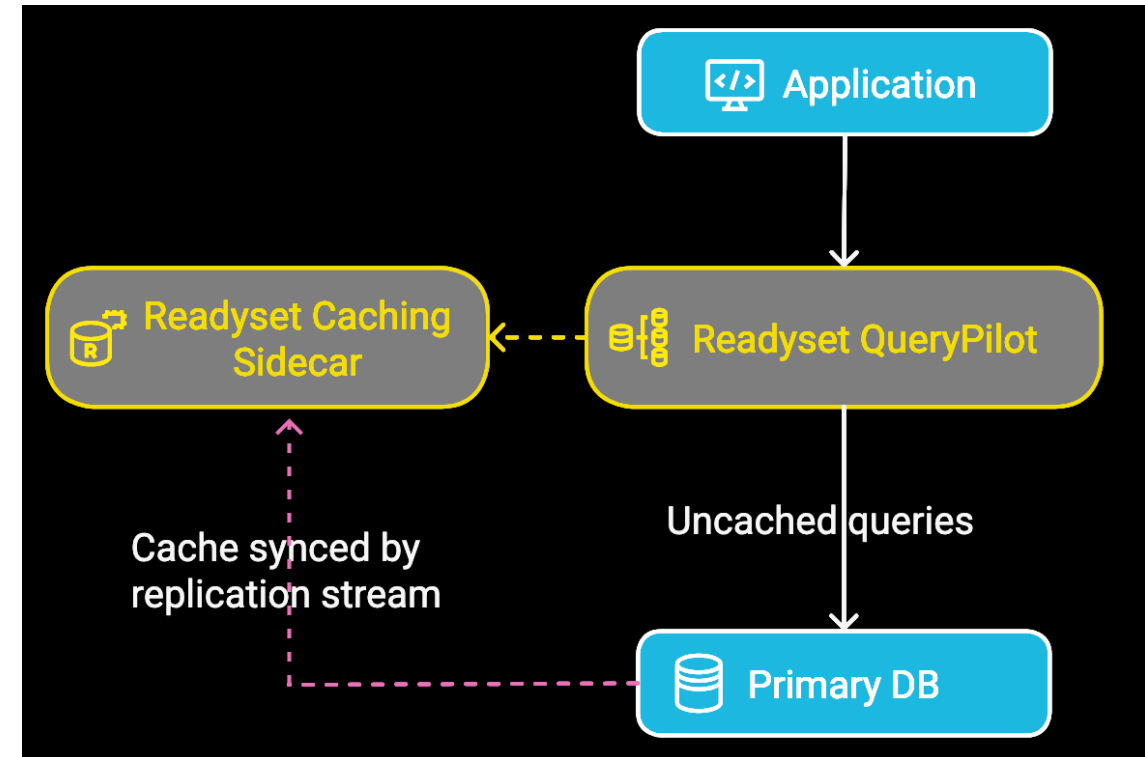


**What if there was a better
way?**

Introducing Readysset QueryPilot for MySQL

Automatic query caching, workload optimization, and routing, eliminating the need for manual tuning and infrastructure management.

- Auto-detection of high-impact queries
- Drop-in deployment for MySQL
- Works with ProxySQL!



Read Replica vs Readysset QueryPilot for MySQL

	Read Replica	 readysset QueryPilot
Query Complexity	Still slow for complex queries	Sub-ms response, automatic caching
Resource Consumption	Linear scaling, high cost per QPS[1]	Non-linear scaling, low cost per QPS
Operational Overhead	High with growing number of replicas	Fewer instances, simpler ops



Demo time!

Try it FREE for 2 months!



Thank you!

Tanmay Sinha

Vice President, Product at Readysset.io



Javier Zon – Reduzindo custos com
Cache Inteligente – Um Guia para DBAs

Time: 10:35 – 11:05